

Understanding Serena in OpenCode

Why Serena is NOT an MCP Server and How It Actually Works

Critical Clarification: Serena is **NOT** an MCP server and **CANNOT** be configured as one. It is OpenCode's built-in code intelligence system that is automatically available when you use OpenCode.

Table of Contents

- 1. [The Misunderstanding](#)
- 2. [What Serena Actually Is](#)
- 3. [Why You See Serena Tools in This Session](#)
- 4. [How to Ensure Serena is Working](#)
- 5. [Difference: Built-in vs MCP](#)
- 6. [Your MCP Servers](#)
- 7. [Verifying Your Setup](#)
- 8. [What You CAN Configure](#)
- 9. [Architecture Explanation](#)
- 10. [FAQ](#)

The Misunderstanding

What You're Asking

"How do I integrate Serena system on OpenCode as an MCP like I have on this session?"

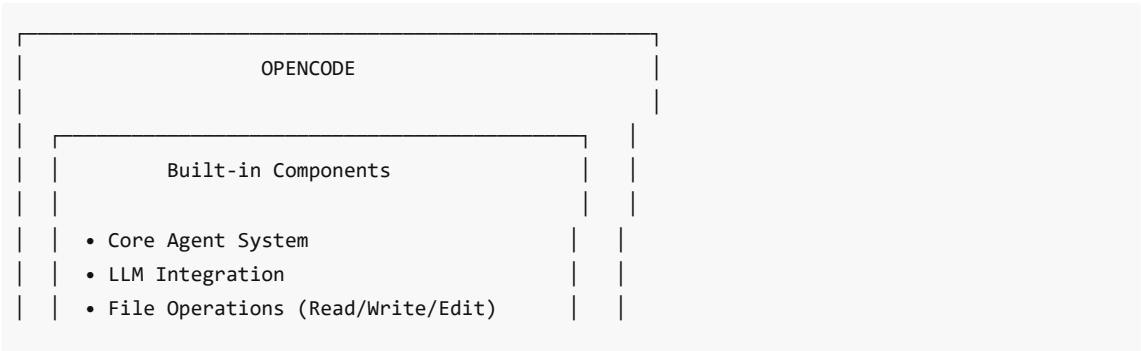
The Reality

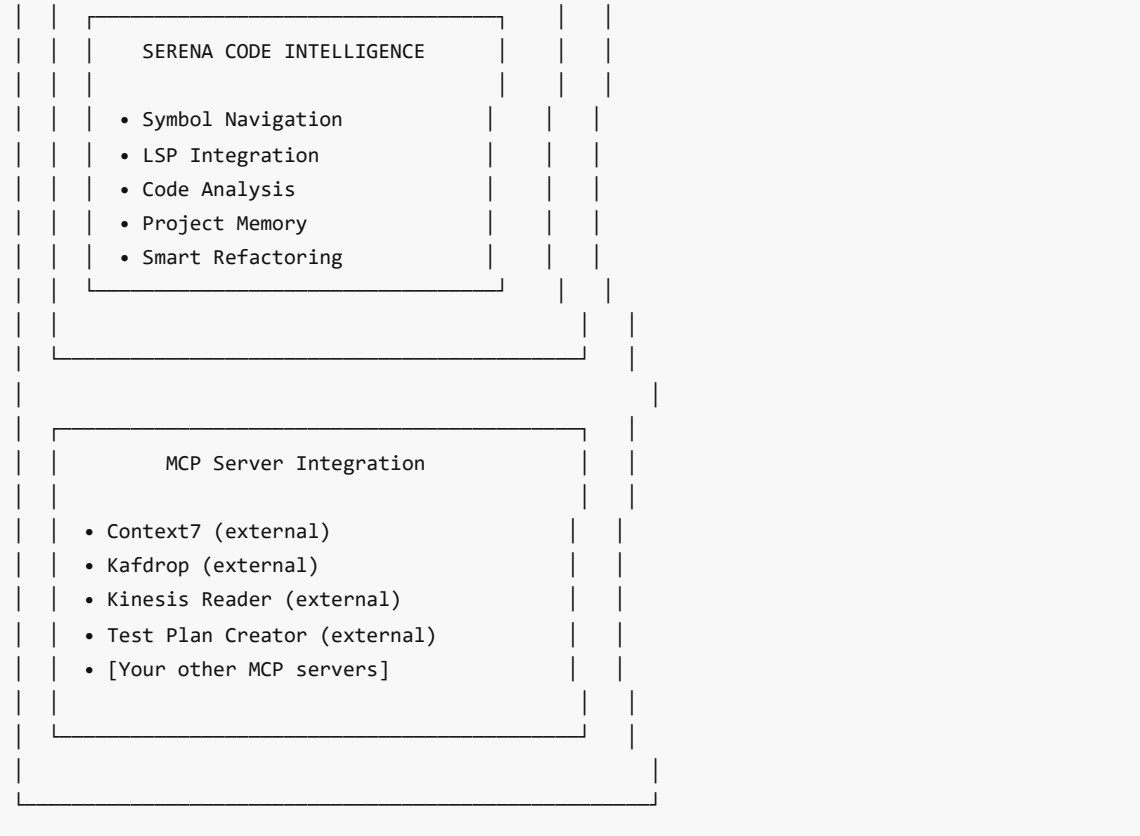
Serena is already integrated - it's not an MCP server. You cannot add Serena as an MCP server because:

- 1. ✓ **Serena is built into OpenCode itself**
- 2. ✓ **It's automatically available in all OpenCode sessions**
- 3. ✓ **No installation or configuration needed**
- 4. ✗ **It's NOT an external MCP server**
- 5. ✗ **You CANNOT add it to your `opencode.json` MCP section**

What Serena Actually Is

Serena's True Nature





Key Points

Aspect	Serena	MCP Servers
Location	Inside OpenCode binary	External processes
Installation	Pre-installed	Must be installed
Configuration	Optional (LSP)	Required in config
Activation	Always available	Must be started
Purpose	Code intelligence	External service access
Examples	Symbol search, refactoring	GitHub, databases, APIs

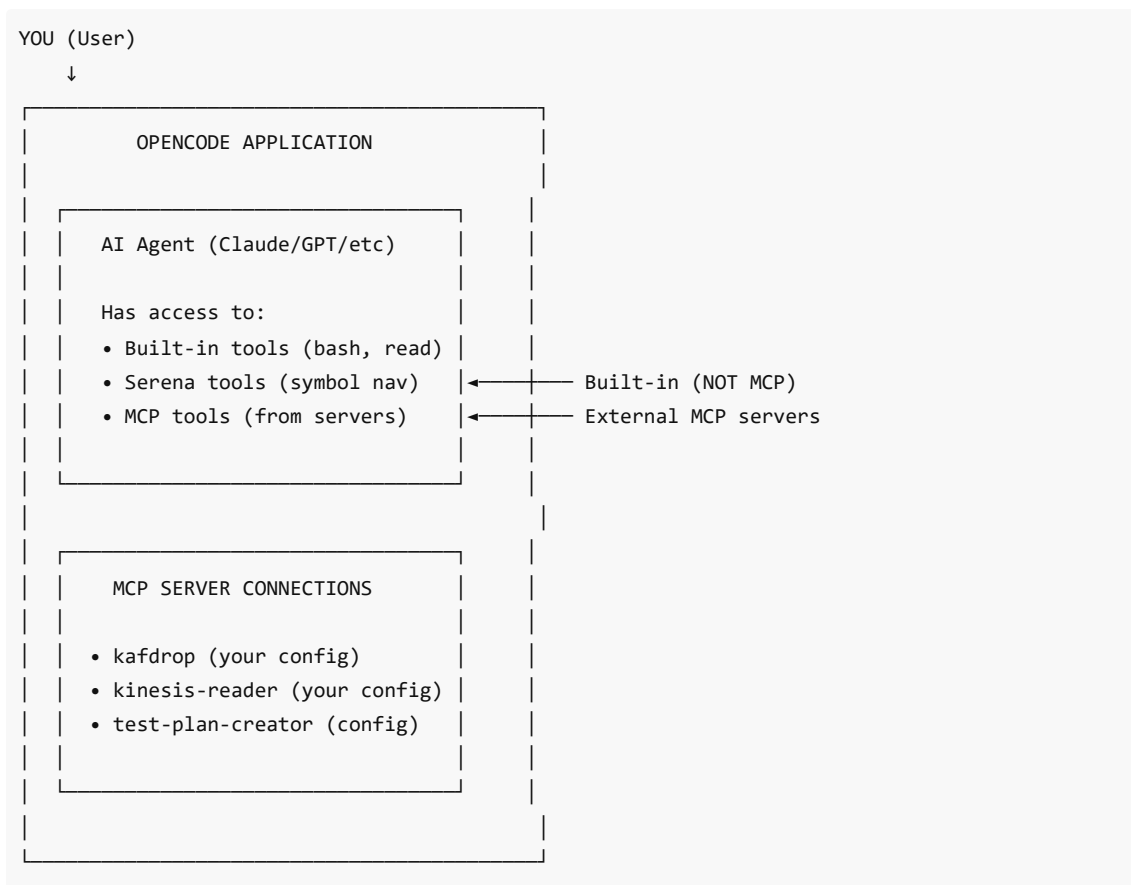
Why You See Serena Tools in This Session

What's Happening

When you see Serena tools in this OpenCode session, it's because:

1. **You're using OpenCode** - The application itself
2. **Serena is part of OpenCode** - Built-in functionality
3. **OpenCode exposes Serena tools to the AI** - Automatic integration
4. **No MCP configuration needed** - It just works

Your Session Architecture



How to Ensure Serena is Working

Since Serena is built-in, you just need to verify it's functioning:

Step 1: Check You're Using OpenCode

```
opencode --version
```

Expected output: `opencode version X.X.X`

If you see this, Serena is available.

Step 2: Test Serena in a Code Project

1. Navigate to a code project:

```
cd /path/to/your/code/project
opencode
```

2. Initialize the project:

```
/init
```

3. Test a Serena query:

```
Show me the structure of [any .ts or .py file]
```

If you get a structured response with classes/functions/methods, Serena is working!

Step 3: Enable LSP (Optional but Recommended)

Create or edit `opencode.json` :

```
{
  "$schema": "https://opencode.ai/config.json",
  "lsp": true
}
```

This enhances Serena's capabilities but is not required.

Difference: Built-in vs MCP

Built-in Systems (Like Serena)

Characteristics:

-  Part of OpenCode binary
-  Always available
-  No installation needed
-  No configuration required (though can be enhanced)
-  Work offline
-  Tightly integrated

Examples in OpenCode:

- **Serena** - Code intelligence
- **bash/shell** - Command execution
- **read/write/edit** - File operations
- **glob/grep** - File search
- **task** - Subprocess management

Access Method: Automatic when using OpenCode

MCP Servers (External Integrations)

Characteristics:

-  External processes
-  Must be installed
-  Require configuration in `opencode.json`
-  May need internet/API keys
-  Loosely coupled

Examples in Your Setup:

- **kafdrop** - Kafka message browsing
- **kinesis-reader** - AWS Kinesis data access
- **test-plan-creator** - JIRA test management

- **Context7** - Documentation search
- **Sentry** - Error tracking

Access Method: Configure in `opencode.json` MCP section

Your MCP Servers

Based on your session, you have these MCP servers configured:

1. Kafdrop (Multiple Environments)

```
{
  "mcp": {
    "kafdrop-dev": {
      "type": "remote",
      "url": "https://kafdrop-dev.example.com/mcp"
    },
    "kafdrop-staging": {
      "type": "remote",
      "url": "https://kafdrop-staging.example.com/mcp"
    },
    "kafdrop-test": {
      "type": "remote",
      "url": "https://kafdrop-test.example.com/mcp"
    }
  }
}
```

2. Kinesis Reader (Multiple Environments)

```
{
  "mcp": {
    "kinesis-reader-dev": {
      "type": "remote",
      "url": "https://kinesis-dev.example.com/mcp"
    },
    "kinesis-reader-staging": {
      "type": "remote",
      "url": "https://kinesis-staging.example.com/mcp"
    },
    "kinesis-reader-test": {
      "type": "remote",
      "url": "https://kinesis-test.example.com/mcp"
    }
  }
}
```

3. Test Plan Creator

```
{
  "mcp": {
    "test-plan-creator": {
      "type": "remote",
      "url": "https://test-plan.example.com/mcp"
    }
  }
}
```

4. Kafdrop Bulk Search

```
{
  "mcp": {
    "kafdrop-bulk-search": {
      "type": "remote",
      "url": "https://kafdrop-bulk.example.com/mcp"
    }
  }
}
```

How to Check Your Actual Configuration

```
# View your OpenCode config
cat ~/.config/opencode/opencode.json

# Or for project-specific
cat ./opencode.json
```

Verifying Your Setup

Check 1: OpenCode Version

```
opencode --version
```

Check 2: Your MCP Servers

```
opencode mcp list
```

Expected output: List of your configured MCP servers (kafdrop, kinesis, etc.)

Check 3: Serena (Built-in)

```
# In OpenCode TUI
Show me available tools
```

You should see:

- ☒ Serena tools (serena_find_symbol, serena_get_symbols_overview, etc.)
- ☒ Built-in tools (bash, read, write, edit, etc.)
- ☒ MCP tools (kafdrop_*, kinesis_*, test-plan-creator_*, etc.)

Check 4: Test Each System

Test Serena (built-in):

```
Find the main function in this project
```

Test MCP server:

```
List Kafka topics using kafdrop
```

Test built-in tools:

```
Show me the current directory
```

What You CAN Configure

Even though Serena is built-in, you can enhance it:

1. Enable LSP

Location: `~/.config/opencode/opencode.json` or project `opencode.json`

```
{
  "$schema": "https://opencode.ai/config.json",
  "lsp": true
}
```

Benefits:

- Better symbol resolution
- Faster searches
- More accurate diagnostics
- Enhanced code completion

2. Configure Language-Specific LSP

```
{
  "$schema": "https://opencode.ai/config.json",
  "lsp": {
    "typescript": {
      "disabled": false
    },
    "python": {
      "disabled": false
    },
    "java": {
```

```
    "disabled": false
  }
}
```

3. Optimize File Watching

```
{
  "$schema": "https://opencode.ai/config.json",
  "watcher": {
    "ignore": [
      "node_modules/**",
      "dist/**",
      "build/**",
      ".git/**",
      "vendor/**"
    ]
  }
}
```

4. Configure Snapshots

```
{
  "$schema": "https://opencode.ai/config.json",
  "snapshot": true
}
```

5. Memory Configuration

Serena's memory is automatic, but you can interact with it:

```
# Write to memory
Remember that we use camelCase for methods

# List memories
/memories list

# Read memory
/memories read pattern-name

# Delete memory
/memories delete old-pattern
```

Architecture Explanation

How OpenCode Really Works



- | |
|---------------------|
| • Kafdrop |
| • Kinesis Reader |
| • Test Plan Creator |
| • Context7 |
| • Sentry |
| • [Others] |

Key Insights

1. Serena lives **INSIDE** the OpenCode process

- Part of the same binary
- Always loaded
- No network calls needed

2. MCP servers are **EXTERNAL** processes

- Separate programs/services
- Connected via HTTP/stdio
- May require network/authentication

3. Both appear as tools to the AI

- AI doesn't distinguish between built-in and external
- All tools have consistent interface
- OpenCode handles the routing

FAQ

Q: Can I install Serena as an MCP server?

A: No. Serena is not an MCP server. It's built into OpenCode itself.

Q: Why do I see Serena tools in my session?

A: Because you're using OpenCode, and Serena is part of OpenCode.

Q: Do I need to configure Serena in `opencode.json`?

A: No configuration required. You CAN enable LSP to enhance it, but Serena works without any config.

Q: Is Serena available in other AI coding tools?

A: No. Serena is unique to OpenCode.

Q: Can I use Serena from outside OpenCode?

A: No. Serena is only available within OpenCode sessions.

Q: How do I enable Serena?

A: You don't. It's always enabled when you use OpenCode.

Q: Can I disable Serena?

A: No. Serena is integral to OpenCode's code intelligence.

Q: Do I need internet for Serena?

A: No. Serena works completely offline.

Q: Can I create my own Serena-like MCP server?

A: Yes, but it would be a separate MCP server, not "Serena". You could create an MCP server with similar functionality.

Q: Why isn't Serena listed when I run `opencode mcp list` ?

A: Because it's not an MCP server. That command only lists external MCP servers.

Q: How do I update Serena?

A: Update OpenCode itself: `npm update -g opencode-ai` or `brew upgrade opencode`

Q: Can other OpenCode users use Serena?

A: Yes, automatically. Every OpenCode installation includes Serena.

Complete Configuration Example

Here's what a complete OpenCode configuration looks like, showing the difference between built-in enhancement (LSP for Serena) and MCP servers:

~/.config/opencode/opencode.json:

```
{
  "$schema": "https://opencode.ai/config.json",

  "// =====": "",
  "// BUILT-IN SERENA ENHANCEMENT (Optional)": "",
  "// =====": "",

  "lsp": true,

  "watcher": {
    "ignore": [
      "node_modules/**",
      "dist/**",
      "build/**",
      ".git/**"
    ]
  },

  "snapshot": true,

  "// =====": "",
  "// EXTERNAL MCP SERVERS (Must be configured)": "",
  "// =====": "",
}
```

```

"mcp": {
  "kafdrop-staging": {
    "type": "remote",
    "url": "https://kafdrop-staging.example.com/mcp",
    "enabled": true,
    "headers": {
      "Authorization": "Bearer {env:KAFDROP_TOKEN}"
    }
  },

  "kinesis-reader-staging": {
    "type": "remote",
    "url": "https://kinesis-staging.example.com/mcp",
    "enabled": true,
    "headers": {
      "Authorization": "Bearer {env:KINESIS_TOKEN}"
    }
  },

  "test-plan-creator": {
    "type": "remote",
    "url": "https://test-plan.example.com/mcp",
    "enabled": true,
    "headers": {
      "Authorization": "Bearer {env:TEST_PLAN_TOKEN}"
    }
  },

  "context7": {
    "type": "remote",
    "url": "https://mcp.context7.com/mcp",
    "enabled": true
  }
},

// =====: "",
// OTHER OPENCODE SETTINGS: "",
// =====: "",

"model": "anthropic/claude-sonnet-4-5",
"autoupdate": true,
"share": "manual"
}

```

What This Shows

Lines for Serena (Built-in):

- "lsp": true - Enhances Serena (optional)
- "watcher.ignore" - Helps Serena ignore files (optional)
- "snapshot": true - Enables undo/redo (optional)

Lines for MCP Servers (External):

- Everything in the "mcp" section
- Each server must be explicitly configured
- Requires URLs, authentication, etc.

Notice:

- ❌ No "serena" in the MCP section
- ❌ No Serena installation commands
- ❌ No Serena URL or authentication
- ✅ Serena just works because it's built-in

Summary: The Truth About Serena

✅ What Serena IS

- Built into OpenCode
- Always available
- Code intelligence system
- No installation needed
- No configuration required (LSP optional)
- Works offline
- Automatic

❌ What Serena is NOT

- Not an MCP server
- Not external software
- Not separately installable
- Not configurable in MCP section
- Not listed in `opencode mcp list`
- Not a service you connect to

🎯 How to "Use Serena"

1. ✅ Install OpenCode
2. ✅ Navigate to a code project
3. ✅ Run `/init`
4. ✅ Start asking code-related questions
5. ✅ Optionally enable LSP for better results

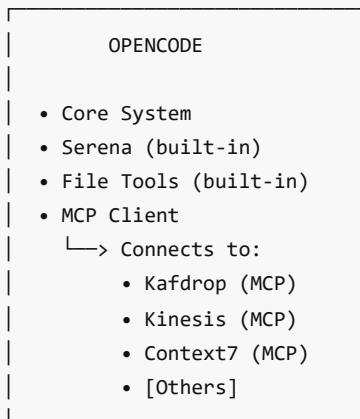
That's it! There is no step 6. Serena is already there.

Visual Comparison

❌ WRONG Mental Model

OpenCode —connects to—> Serena MCP Server
(doesn't exist)

✅ CORRECT Mental Model



Conclusion

You cannot integrate Serena as an MCP server because Serena is already integrated as a core component of OpenCode.

When you use OpenCode, you automatically get:

- ☒ Serena code intelligence
- ☒ Built-in file operations
- ☒ Shell execution
- ☒ PLUS your configured MCP servers

No additional setup needed for Serena!

Appendix: If You Want MCP-Like Code Intelligence

If you want to create an **external** MCP server that provides code intelligence (separate from Serena), you could:

Option 1: Create Custom MCP Server

Build an MCP server using:

- Python MCP SDK: <https://github.com/modelcontextprotocol/python-sdk>
- TypeScript MCP SDK: <https://github.com/modelcontextprotocol/typescript-sdk>

Implement tools for:

- Code search
- Symbol navigation
- Diagnostics
- Refactoring

Option 2: Use Existing MCP Servers

Some MCP servers provide code-related functionality:

- **GitHub MCP** - Repository access
- **GitLab MCP** - Project management
- **Filesystem MCP** - File operations

But these are complementary to Serena, not replacements.

Document Information:

- **Version:** 1.0
- **Last Updated:** May 15, 2026
- **OpenCode Compatibility:** 1.3.0+
- **Critical Point:** Serena is NOT an MCP server

Key Takeaway: Stop looking for how to "install" or "configure" Serena as an MCP server. Instead, just **use OpenCode**

- Serena is already there, working silently in the background, providing code intelligence automatically!